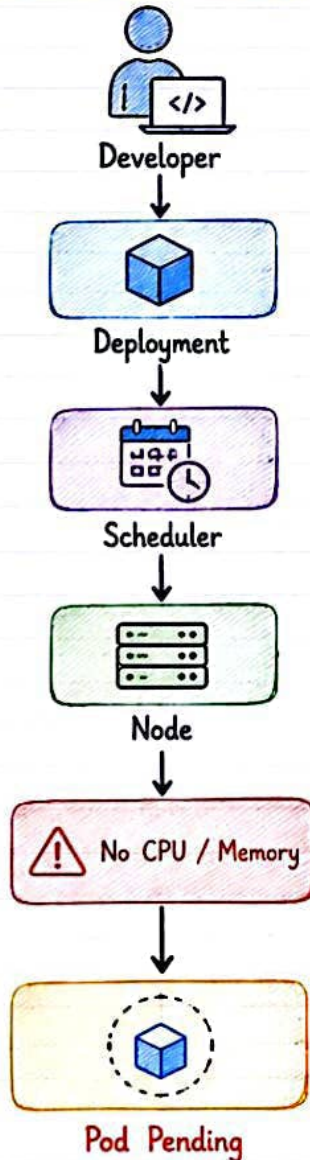


1. Kubernetes Pods Stuck in Pending



kubernetes

ARCHITECTURE



SYMPTOMS

- ✗ Pods remain in Pending
- ✗ Application unavailable



ROOT CAUSE

- Insufficient CPU
- Insufficient Memory
- Node Selector mismatch
- Taints
- PVC Pending



TROUBLESHOOTING

- | | |
|--|------------------------|
| <code>kubectl get pods</code> | # List all pods |
| <code>kubectl describe pod <pod-name></code> | # Describe pod |
| <code>kubectl get nodes</code> | # List nodes |
| <code>kubectl top nodes</code> | # Check resource usage |



SOLUTION

- ✓ Add Node Pool
- ✓ Increase VM Size
- ✓ Fix Requests/Limits
- ✓ Remove Wrong NodeSelector
- ✓ Verify PVC



REMEMBER

- Always set proper Requests and Limits
- Monitor Node resources continuously
- Use Cluster Autoscaler
- Check Events for exact error
- Validate PVC and Storage Class



BEST PRACTICES

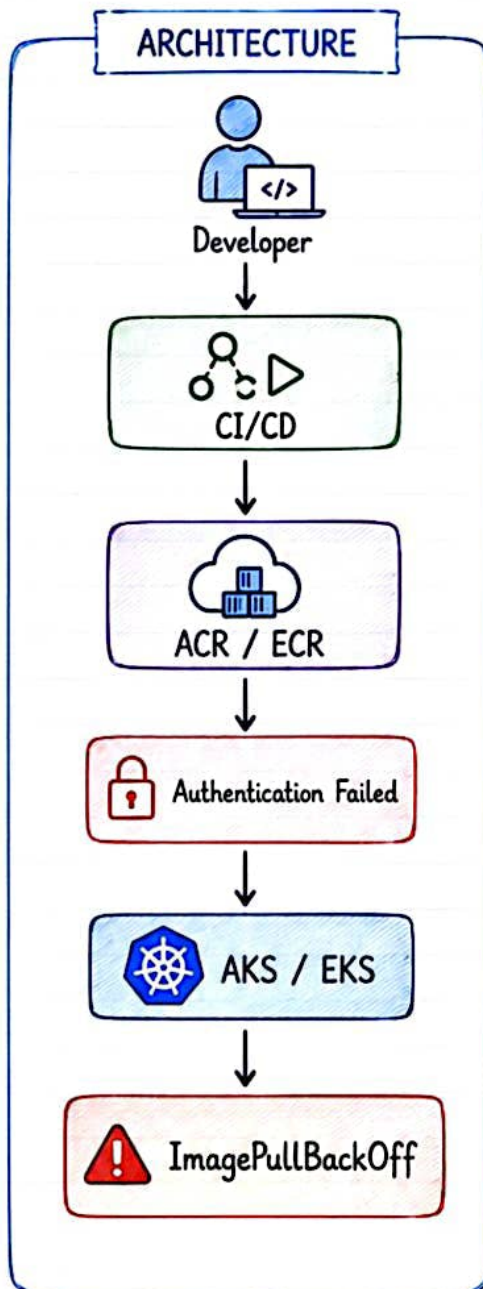
- ✓ Right-size your workloads
- ✓ Use HPA and Cluster Autoscaler
- ✓ Set Resource Quotas
- ✓ Use Health Checks and Alerts
- ✓ Regularly review and optimize resources





kubernetes

2. ImagePullBackOff



ROOT CAUSE

- Wrong Image Name
- Wrong Tag
- Registry Permission
- Registry Down
- Expired Secret



SOLUTION

- ✓ Verify Image
- ✓ Check Registry Login
- ✓ Update Secret
- ✓ Restart Deployment



TROUBLESHOOTING COMMANDS

- | | |
|---|-------------------------|
| • <code>kubectl get pods</code> | # Check pod status |
| • <code>kubectl describe pod <pod-name></code> | # Check events & reason |
| • <code>kubectl get secret</code> | # List secrets |
| • <code>kubectl get events --sort-by=.metadata.creationTimestamp</code> | # Recent events |



REMEMBER

- ★ ImagePullBackOff means container image cannot be pulled.
- ★ Always use correct image name and tag.
- ★ Use imagePullSecrets for private registries.
- ★ Check network connectivity to registry.



BEST PRACTICES

- ★ Use valid and trusted container images
- ★ Store registry credentials securely
- ★ Automate image scans in CI/CD
- ★ Use specific image tags (avoid latest)
- ★ Monitor registry availability



Kubernetes



Azure AKS



AWS EKS



Docker



Jenkins



GitHub Actions



Helm



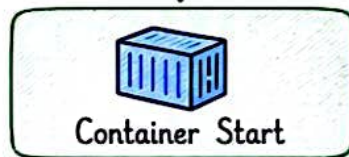
ACR / ECR



kubernetes

3. CrashLoopBackOff

ARCHITECTURE



ROOT CAUSE

- Wrong ENV Variable
- Database Down
- Application Bug
- Port Conflict
- Secret Missing



SOLUTION

- ✓ Check Logs
- ✓ Verify ConfigMap
- ✓ Verify Secret
- ✓ Fix Application



TROUBLESHOOTING COMMANDS

- `kubectl get pods` # List all pods
- `kubectl describe pod <pod-name>` # Describe pod
- `kubectl logs <pod-name>` # Check pod logs
- `kubectl get events --sort-by=.metadata.creationTimestamp` # Recent events



REMEMBER

- ★ CrashLoopBackOff means container is crashing repeatedly.
- ★ Always check logs first.
- ★ ENV, ConfigMaps and Secrets are common misconfigurations.
- ★ Dependency (DB, External Service) down also causes crash.

Logs are your best friend!



BEST PRACTICES

- ★ Validate configuration before deployment.
- ★ Use liveness and readiness probes.
- ★ Monitor logs and set alerts.
- ★ Automate rollback for failed deployments.



Kubernetes



Helm



Docker



Prometheus



Grafana



Azure AKS



AWS EKS

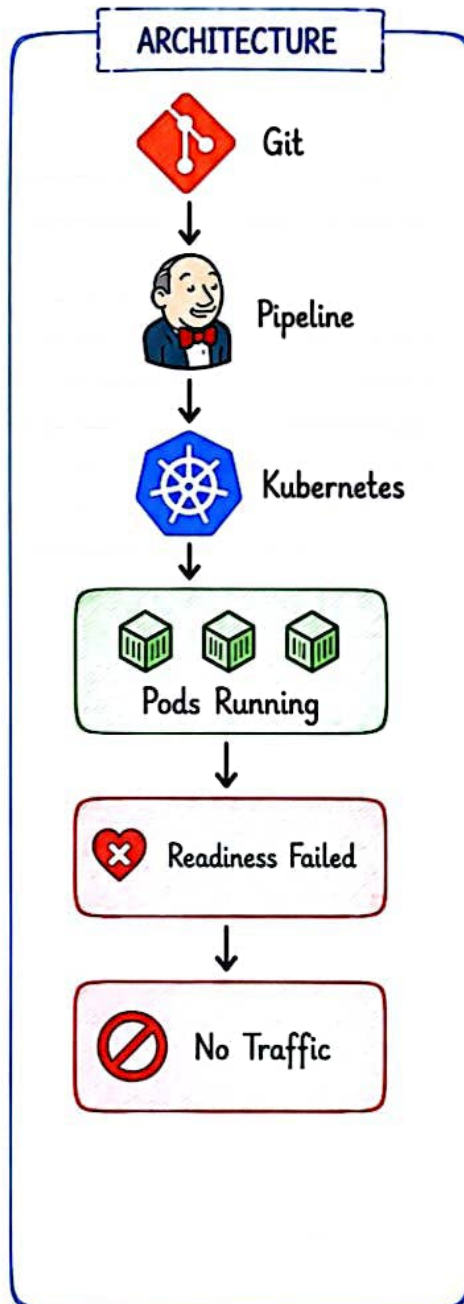


GitHub Actions



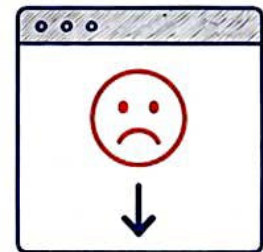
kubernetes

4. Application Down After Deployment



ROOT CAUSE

- Readiness Probe
- Wrong Health Endpoint
- Wrong Port
- Service Selector

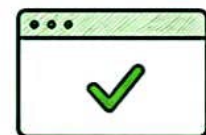


503 / Unavailable



SOLUTION

- ✓ Fix Probe
- ✓ Check Service
- ✓ Validate Endpoint



Traffic OK



TROUBLESHOOTING COMMANDS

- `kubectl get pods` # Check pods status
- `kubectl describe pod <pod-name>` # Check events
- `kubectl get svc` # Check services
- `kubectl get endpoints` # Check endpoints
- `kubectl describe svc <svc-name>` # Check selector



REMEMBER

- ★ Readiness probe failure = Pod is running but not ready.
- ★ Service will not send traffic to unready pods.
- ★ Always test new deployment in lower environment first.
- ★ Monitor events and probe logs.



BEST PRACTICES

- ★ Configure proper Readiness & Liveness probes.
- ★ Use meaningful health endpoints.
- ★ Always validate Service selector.
- ★ Enable monitoring and alerts for failed deployments.



Git



Jenkins



Kubernetes



Helm



Prometheus



Grafana



NGINX



K8s
Dashboard



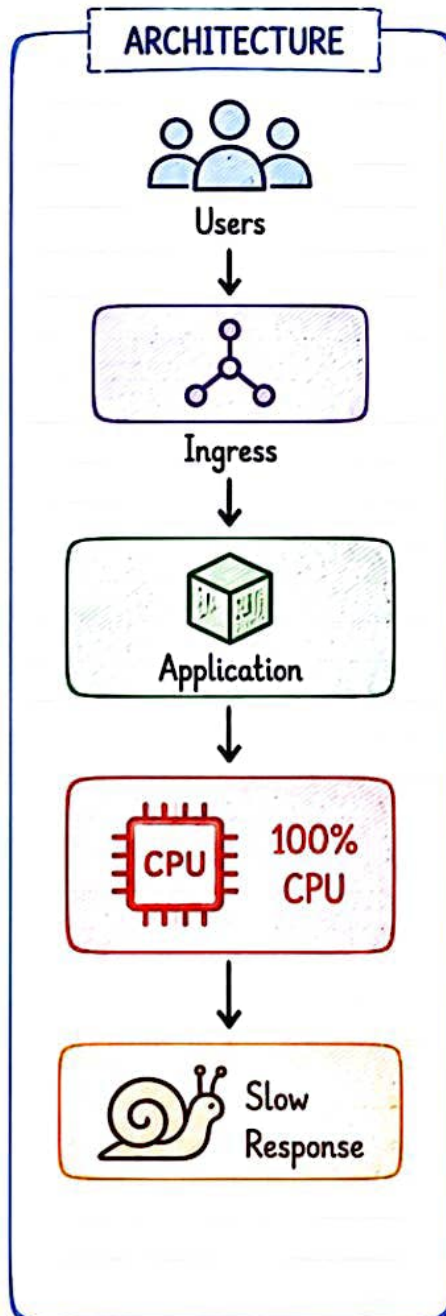
Container



kubernetes

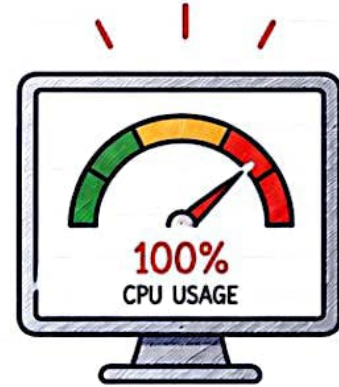
5. High CPU Usage

ARCHITECTURE



ROOT CAUSE

- Infinite Loop
- Heavy Query
- Memory Leak
- Traffic Spike



SOLUTION

- ✓ HPA (Horizontal Pod Autoscaler)
- ✓ Autoscaler (Cluster Autoscaler)
- ✓ Optimize Code
- ✓ Increase Resources (CPU / Memory)



TROUBLESHOOTING COMMANDS

- `kubectl top pods --sort-by=cpu` # Top CPU using pods
- `kubectl top nodes` # Top CPU using nodes
- `kubectl describe pod <pod-name>` # Describe pod
- `kubectl logs <pod-name>` # Check logs
- `kubectl get hpa` # Check HPA status



REMEMBER

- ★ High CPU directly impacts user experience.
- ★ Always check CPU usage trend before scaling.
- ★ Optimize application before adding more resources.
- ★ Set proper Requests & Limits for containers.

Monitor Continuously
Act Quickly
Prevent Downtime



BEST PRACTICES

- ★ Use HPA and Cluster Autoscaler.
- ★ Set Resource Requests & Limits.
- ★ Optimize slow queries and code.
- ★ Enable monitoring and alerts.
- ★ Perform load testing regularly.



Kubernetes



HPA



Prometheus



Grafana



Helm



NGINX

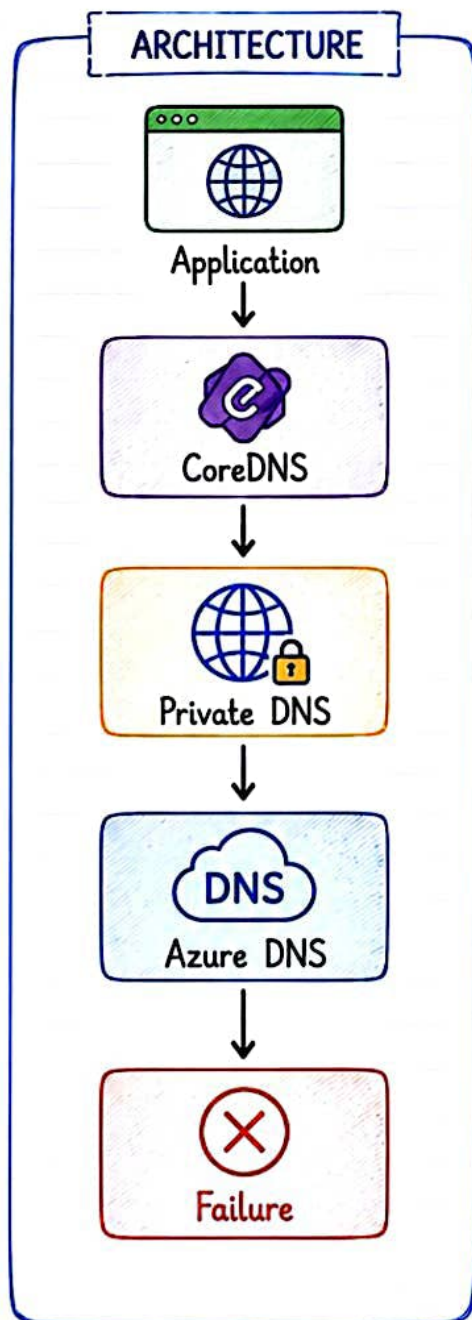


Docker



kubernetes

6. DNS Resolution Failure



ROOT CAUSE

- CoreDNS
- DNS Zone
- Private Endpoint
- Firewall



SOLUTION

- ✓ Restart CoreDNS
- ✓ Verify DNS Zone
- ✓ Check Resolver
- ✓ nslookup



TROUBLESHOOTING COMMANDS

- `kubectl get pods -n kube-system` # Check CoreDNS pods
- `kubectl logs -n kube-system -l k8s-app=kube-dns` # CoreDNS logs
- `nslookup <hostname>` # DNS lookup test
- `dig <hostname>` # Detailed DNS query
- `kubectl get svc -n kube-system` # CoreDNS Service



REMEMBER

- ★ DNS issues can impact entire application.
- ★ Always check CoreDNS first during DNS failure.
- ★ Private DNS misconfiguration is a common cause.
- ★ Validate DNS resolution from inside the cluster.



BEST PRACTICES

- ★ Monitor CoreDNS health and logs.
- ★ Use Private DNS Zones correctly.
- ★ Keep DNS records up to date.
- ★ Test DNS resolution during deployments.
- ★ Use Alerts for CoreDNS failures.



Kubernetes



CoreDNS



Azure DNS



Private DNS



Private Endpoint



Firewall

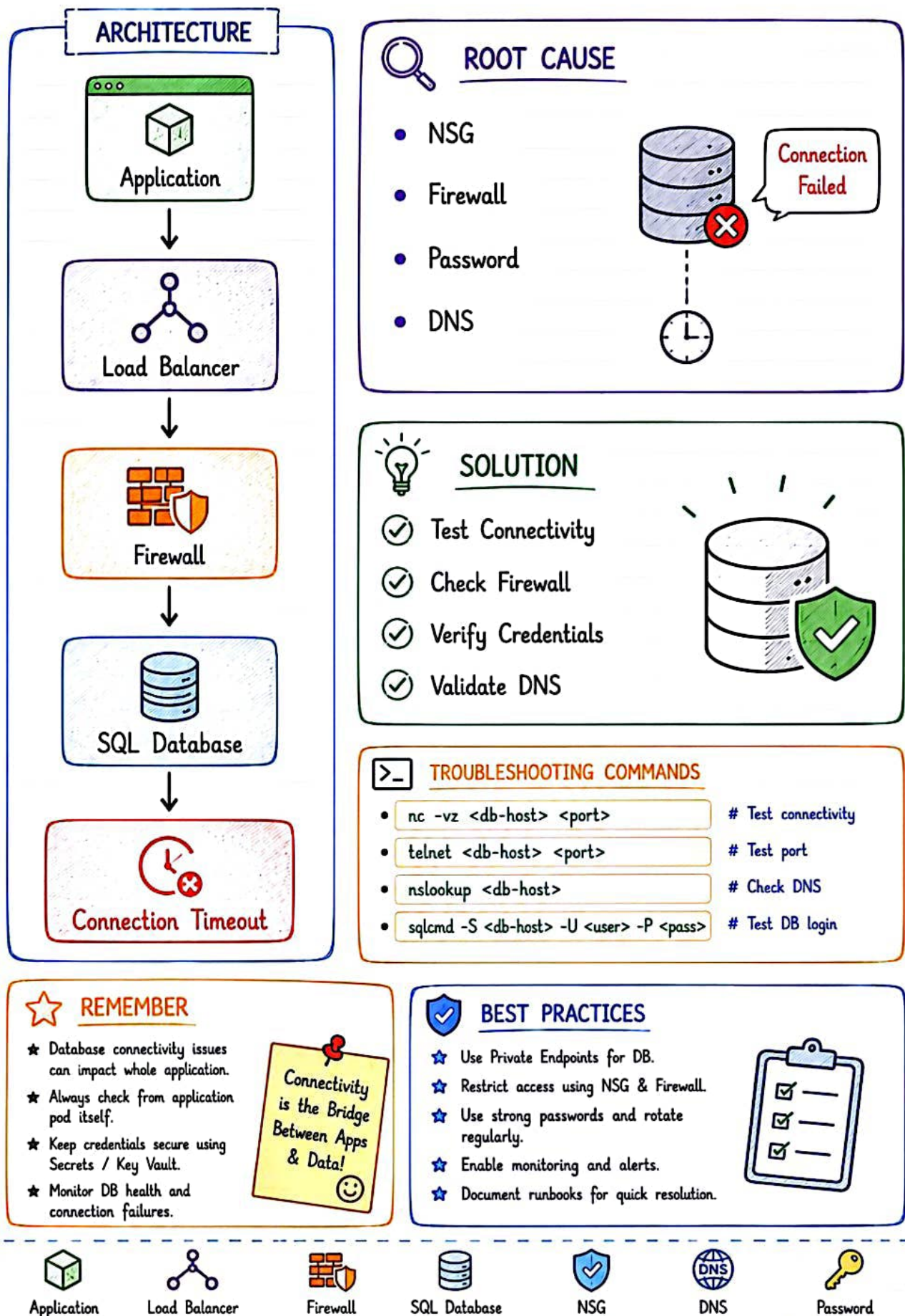


DNS



kubernetes

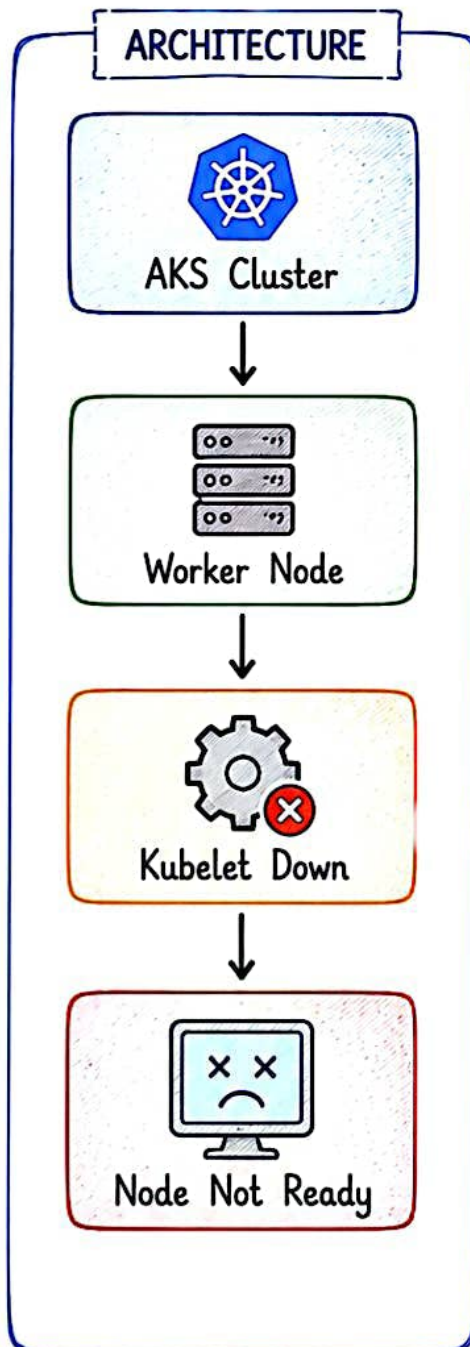
7. Database Connection Failure





kubernetes

8. Node Not Ready



ROOT CAUSE

- Disk Full
- Network Issue
- Kubelet Crash
- VM Failure



SOLUTION

- ✓ Restart Node
- ✓ Restart Kubelet
- ✓ Check Disk
- ✓ Reimage Node



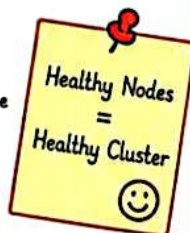
TROUBLESHOOTING COMMANDS

- | | |
|--|----------------------|
| <code>kubectl get nodes</code> | # Check node status |
| <code>kubectl describe node <node-name></code> | # Describe node |
| <code>journalctl -u kubelet -xe</code> | # Check kubelet logs |
| <code>df -h</code> | # Check disk usage |
| <code>ping <node-ip></code> | # Check network |



REMEMBER

- ★ A single node down may impact application availability.
- ★ Monitor node health and disk usage continuously.
- ★ Set alerts for Kubelet down and node not ready.
- ★ Auto-repair or replace unhealthy nodes.



BEST PRACTICES

- ★ Enable monitoring and alerts.
- ★ Keep nodes updated and patched.
- ★ Use auto-scaling node pools.
- ★ Regularly check disk, memory and kubelet health.
- ★ Have runbooks for quick recovery.



Kubernetes



AKS



VM



Kubelet



Disk



Network



Monitoring

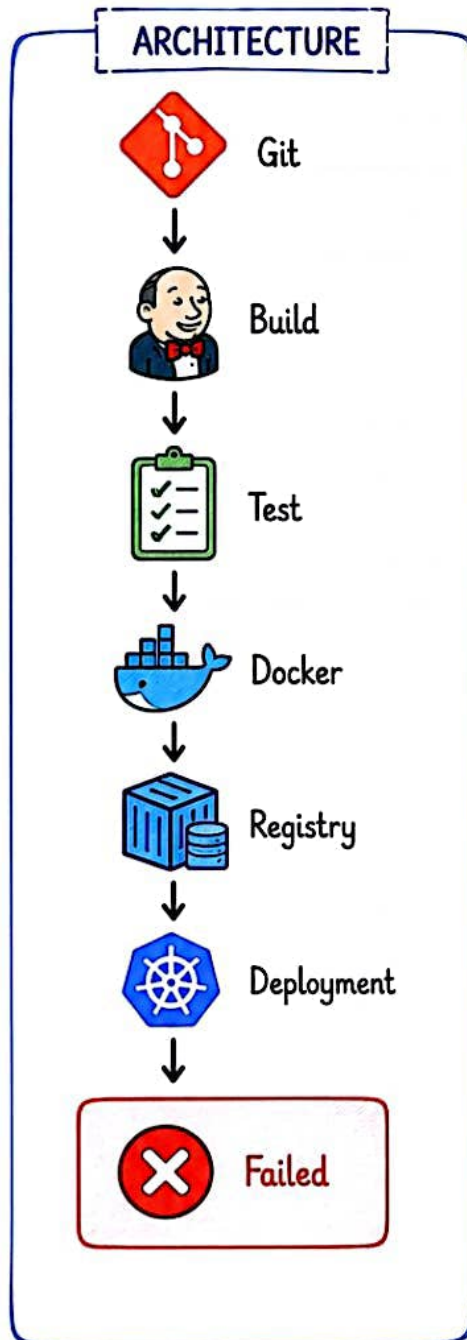


Alerts

9. CI/CD Pipeline Failure

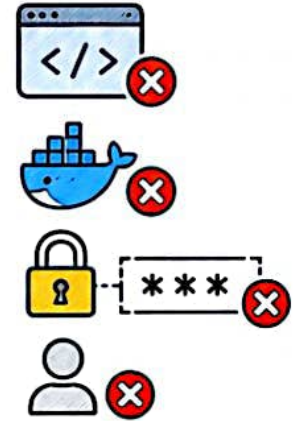


kubernetes



ROOT CAUSE

- Build Error
- Docker Failure
- Secret Missing
- Permission



SOLUTION

- ✓ Check Logs
- ✓ Retry Stage
- ✓ Fix Build
- ✓ Validate Credentials



TROUBLESHOOTING COMMANDS

- `kubectl get pods -n <namespace>` # Check pods
- `kubectl logs <pod-name>` # Check logs
- `kubectl describe pod <pod-name>` # Describe pod
- `kubectl get events -n <namespace>` # Check events
- `kubectl get deployments -n <namespace>` # Check deployments



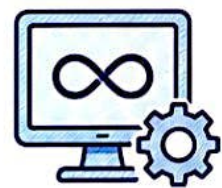
REMEMBER

- ★ Pipeline failure can stop delivery to production.
- ★ Logs are your best friend.
- ★ Automate notifications for quick response.
- ★ Fix root cause, don't just re-run the pipeline.



BEST PRACTICES

- ★ Use version control for all changes.
- ★ Automate build, test & deploy.
- ★ Store secrets securely (Key Vault / Vault / Sealed Secrets).
- ★ Use least privilege access.
- ★ Monitor pipeline and set alerts.



Git



Jenkins



Docker



Kubernetes



Helm



Argo CD



SonarQube



Slack

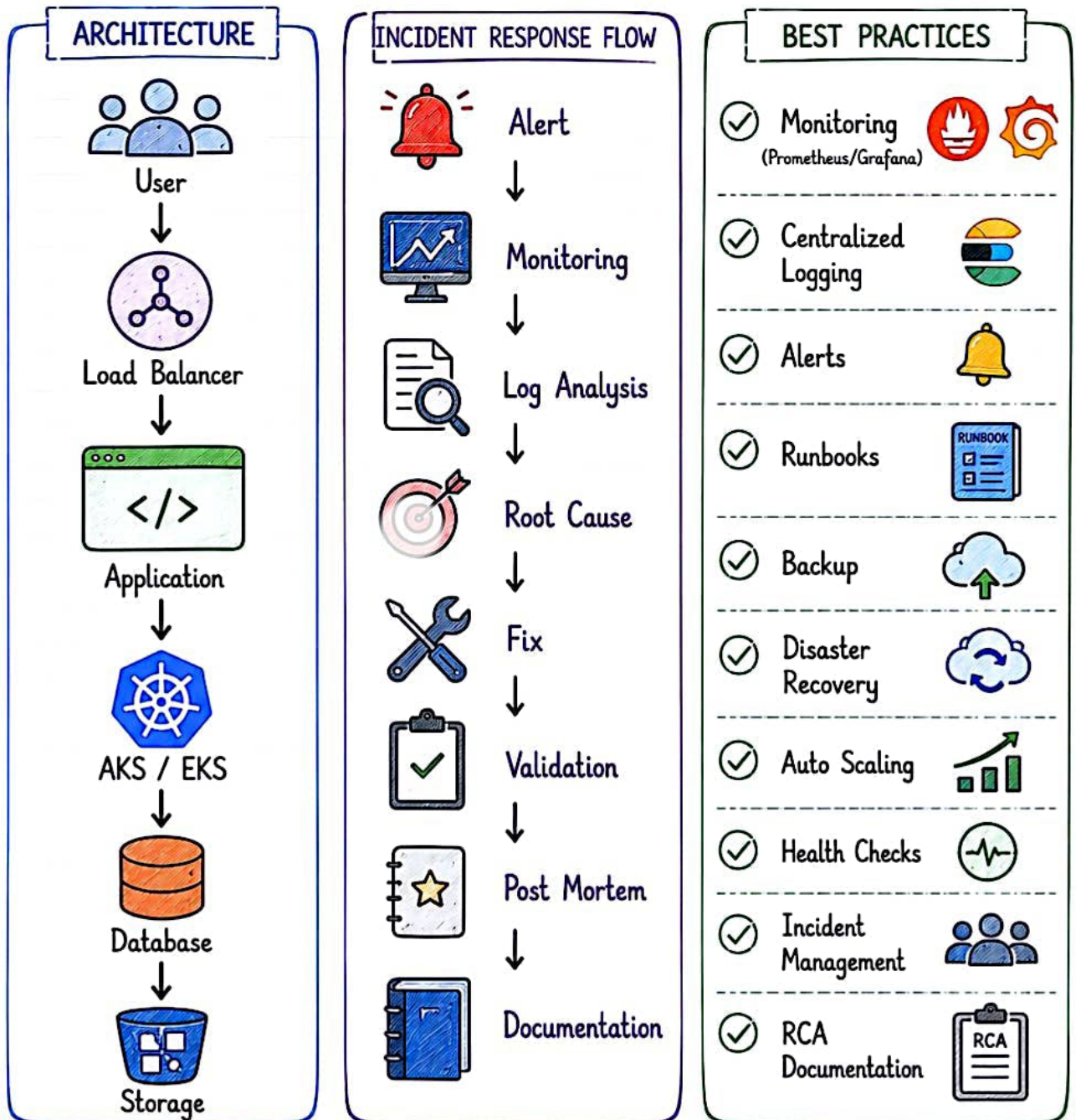


Prometheus

10. Complete Production Incident Flow



kubernetes



KEY POINTS

- ★ Quick detection reduces impact.
- ★ Follow a structured incident response flow.
- ★ Communicate and document everything.
- ★ Learn from incidents and improve continuously.
- ★ Automation + Monitoring = Stable Production.

Plan
Prepare
Respond
Recover
Improve



REMEMBER

- Every incident is an opportunity to improve.
- Good monitoring + logs save hours of troubleshooting.
- Document today, save tomorrow.

Be
Proactive
not
Reactive



Prometheus



Grafana



Loki



Elasticsearch



Kubernetes



Helm



Argo CD



Istio



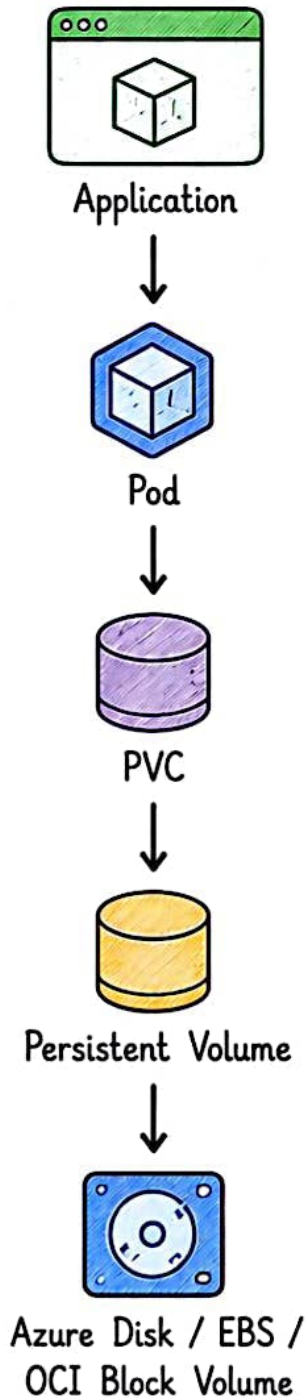
Velero

11. Persistent Volume (PV/PVC) Mount Failure



kubernetes

ARCHITECTURE



SYMPTOMS

- ✗ Pod stuck in ContainerCreating
- ✗ Volume Mount Failed



ROOT CAUSE

- PVC Pending
- Storage Class mismatch
- Volume Attachment Failure
- CSI Driver Issue
- Disk in another Availability Zone



SOLUTION

- ✓ Check PVC
- ✓ Verify StorageClass
- ✓ Restart CSI Driver
- ✓ Reattach Disk
- ✓ Check Zone Mapping



PVC



PV



Disk



CSI Driver



Availability Zone

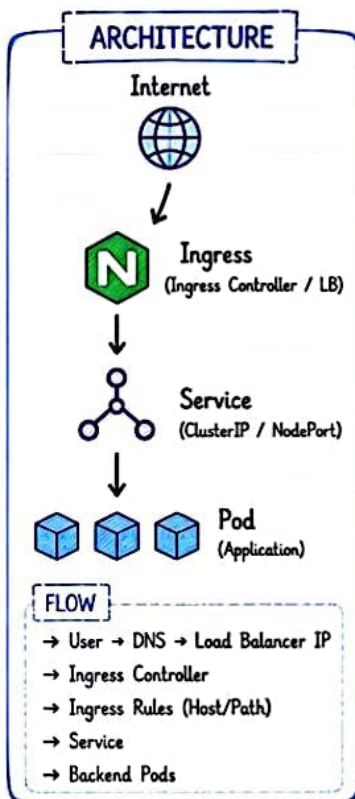


Kubernetes



kubernetes

12. Ingress / Load Balancer Not Routing Traffic



ROOT CAUSE

- Wrong Ingress Class
- Incorrect Host Rule
- TLS Certificate Issue
- Service Port Mismatch
- Backend Pod Unhealthy



SOLUTION (DETAILED)

- ✓ Verify Ingress Resource (kubectl get ingress)
- ✓ Check Ingress Controller Status
- ✓ Validate Ingress Class (nginx / agic / alb / traefik)
- ✓ Verify Hostname & Path Rules in Ingress
- ✓ Check Backend Service Mapping in Ingress
- ✓ Validate Service Port & TargetPort
- ✓ Confirm Pod Labels Match Service Selector
- ✓ Verify Endpoints (kubectl get endpoints)
- ✓ Verify TLS Secret & Certificate Validity
- ✓ Review Ingress Controller Logs
- ✓ Check DNS Resolution & Public IP
- ✓ Test Connectivity Directly from Pod
- ✓ Restart Ingress Controller (if required)
- ✓ Perform End-to-End Traffic Validation

TROUBLESHOOTING COMMANDS

- | | | |
|--|--|---|
| <ul style="list-style-type: none"> • kubectl get ingress • kubectl describe ingress <ingress-name> • kubectl get ingressclass • kubectl describe ingressclass <class-name> • kubectl get svc • kubectl describe svc <service-name> | <ul style="list-style-type: none"> • kubectl get endpoints • kubectl describe endpoints • kubectl get pods -l <label> -o wide • kubectl logs -n ingress-nginx deploy/ingress-nginx-controller • kubectl describe pod <pod-name> • kubectl get pods | <ul style="list-style-type: none"> • curl -H "Host: app.example.com" http://<LB-IP> • curl -k https://app.example.com • nslookup app.example.com • dig app.example.com • openssl s_client -connect app.example.com:443 |
|--|--|---|



COMMON SYMPTOMS

- 404 Not Found
- 502 Bad Gateway
- 503 Service Unavailable
- 504 Gateway Timeout
- No response / Connection timeout
- SSL/TLS handshake failure



BEST PRACTICES

- ★ Use a single Ingress Controller per ingress class.
- ★ Configure Readiness & Liveness Probes for Pods.
- ★ Use TLS with auto-renewal (Cert-Manager / Key Vault / ACM).
- ★ Enable Access & Error Logs.
- ★ Monitor 4xx/5xx response codes and latency.
- ★ Implement Health Checks for backend services.
- ★ Use Canary / Blue-Green deployments.
- ★ Validate DNS after every deployment.
- ★ Keep Ingress Controller & chart updated.
- ★ Use WAF (NGINX App Protect / Azure WAF / AWS WAF) for production.



QUICK CHECKLIST

- ✓ Is Ingress Created with correct class?
- ✓ Is Hostname and Path rule correct?
- ✓ Is Service available and mapped correctly?
- ✓ Are Endpoints populated?
- ✓ Are Backend Pods Running and Healthy?
- ✓ Is TLS secret valid and not expired?
- ✓ Can you access the service from inside the cluster?
- ✓ Can you access the service from outside the cluster?



LOGS TO CHECK

- ★ Ingress Controller Logs (NGINX / AGIC / ALB)
- ★ Application Pod Logs
- ★ Service / Endpoint Events
- ★ Kubernetes Events (kubectl get events --sort-by=.metadata.creationTimestamp)
- ★ DNS / WAF / Load Balancer Logs (Azure / AWS / GCP)



Internet



Ingress



Service



Pod



Endpoints



TLS / Cert



Kubernetes

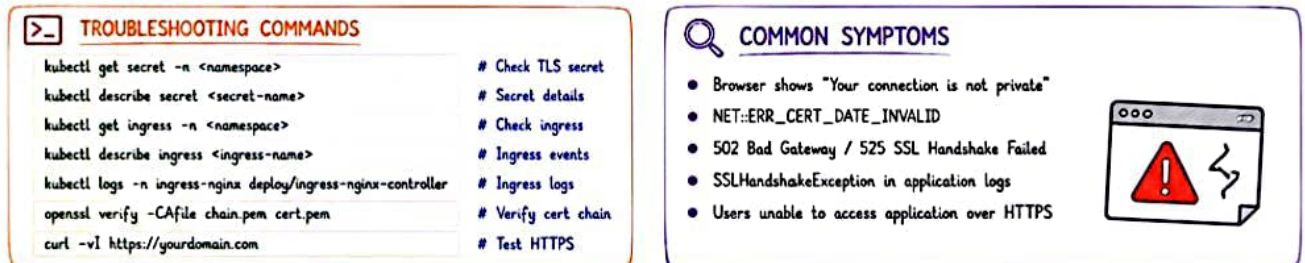
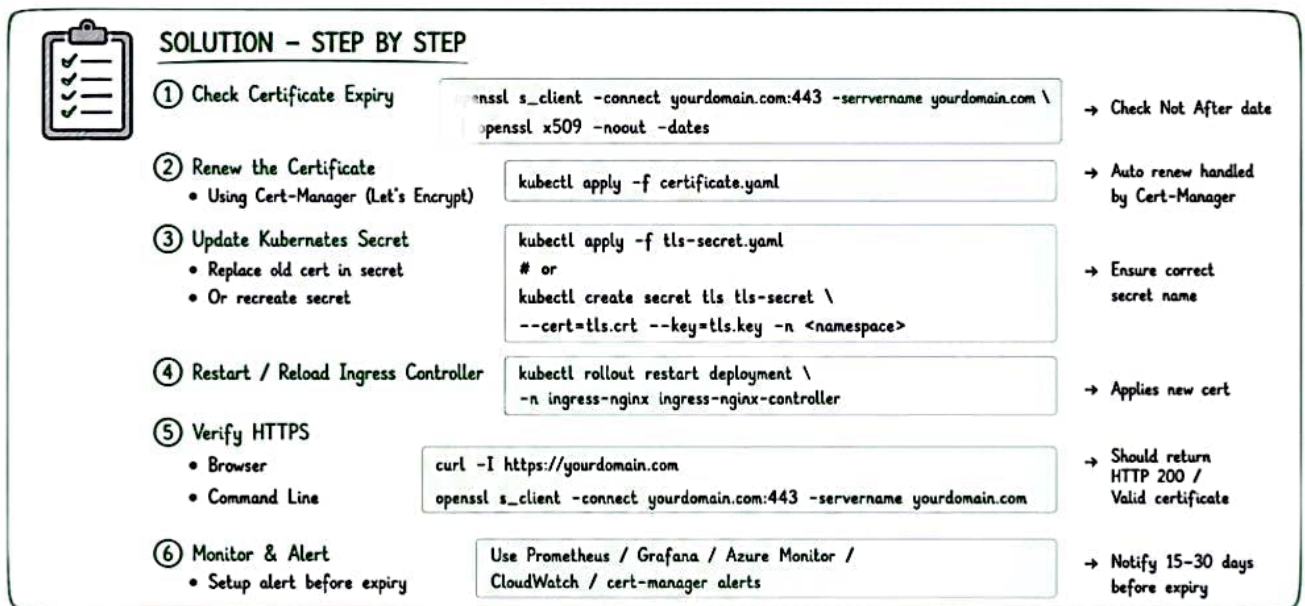
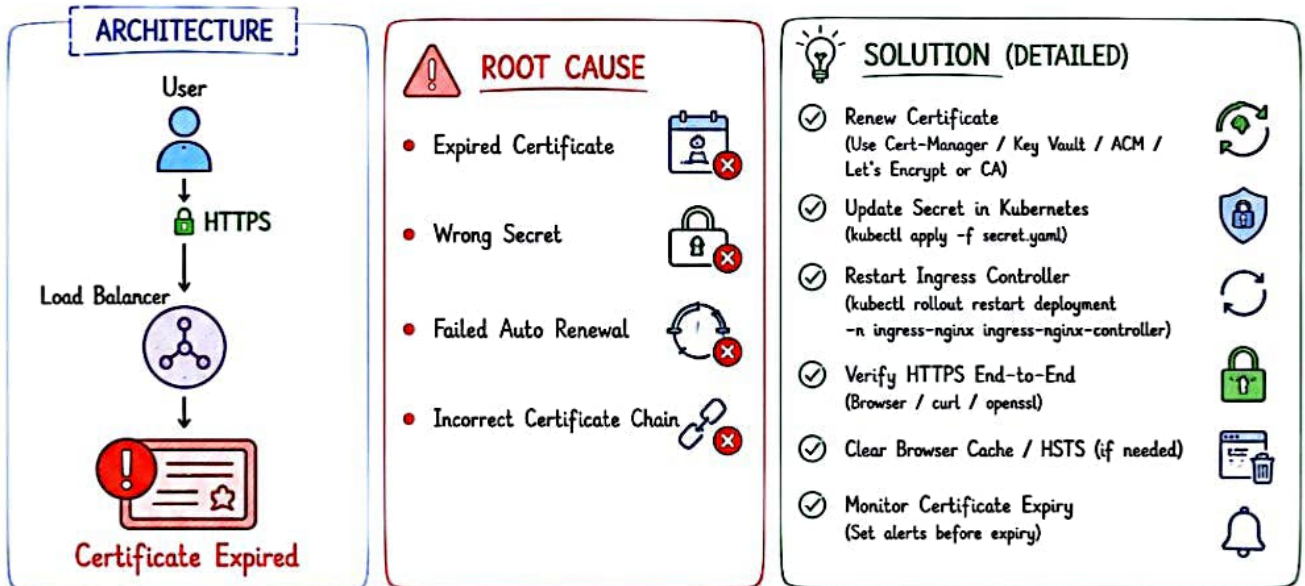


WAF



kubernetes

13. TLS / SSL Certificate Expired

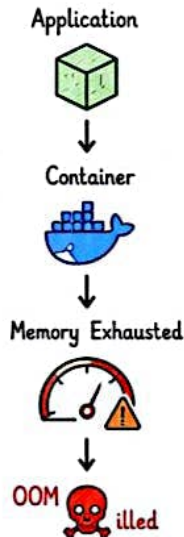


14. Container OOMKilled (Out of Memory)



kubernetes

ARCHITECTURE



ROOT CAUSE

- Memory Leak



- Low Memory Limit



- Large Heap



- Traffic Burst



SOLUTION (DETAILED)

- ✓ Increase Memory
 - Increase limits.memory in Deployment/StatefulSet
 - Ensure node has enough allocatable memory
- ✓ Tune JVM (for Java Applications)
 - Set proper -Xmx (e.g. -Xmx70% of container limit)
 - Use G1GC / ZGC and enable GC logging
- ✓ Optimize Code
 - Fix memory leaks, close connections/streams
 - Avoid loading huge data in memory
 - Use pagination and caching
- ✓ Configure VPA (Vertical Pod Autoscaler)
 - Enable VPA in 'Auto' or 'Recommend' mode
 - Helps right-size memory requests/limits automatically
- ✓ Other Actions
 - Use liveness/readiness probes
 - Enable resource requests & limits
 - Scale horizontally using HPA if needed



HOW OOMKILLED HAPPENS ?

- Container memory usage exceeds the limit
- Linux Kernel (cgroup OOM Killer) terminates the process
- Kubernetes restarts the container based on restartPolicy
- If it keeps happening → Application becomes unstable



Exceeds
Memory Limit



Kernel OOM Killer

Process Terminated



Kubernetes Restarts Container



COMMON SYMPTOMS

- Pod status shows: **OOMKilled**
- Container restarts frequently
- Application crashes under load
- High memory usage before crash
- Events show: "OOMKilled"

```

kubectl get pods
NAME
my-app-7d6d4d8f6c-abc12 0/1 OOMKilled

kubectl describe pod <pod-name>
Last State: Terminated
Reason: OOMKilled
Exit Code: 137
  
```



TROUBLESHOOTING COMMANDS

- `kubectl top pods` # Check current usage
- `kubectl describe pod <pod-name>` # Check events
- `kubectl logs <pod-name> --previous` # Check last logs
- `kubectl get events --sort-by=.lastTimestamp` # Check OOM events
- `kubectl exec -it <pod-name> -- sh` # Inspect container
- `dmesg | grep -i oom` # Node OOM events



SOLUTION - STEP BY STEP

1 Check Memory Usage

```

kubectl top pods
kubectl top nodes
  
```

→ Identify pods with high memory consumption

2 Review Limits & Requests

```

kubectl get pod <pod-name> -o yaml | grep -A 10 resources
  
```

Ensure requests < limits & appropriate values

3 Increase Memory Limit

```

Update the Deployment/StatefulSet to increase limits.memory
  
```

→ Redeploy the application

4 Tune Application / JVM

```

Adjust JVM heap (-Xmx), GC, threads, cache size, connection pool
  
```

→ Reduce unnecessary memory usage

5 Configure VPA

```

kubectl apply -f vpa.yaml
  
```

→ Let VPA recommend / set correct memory

6 Monitor & Validate

```

Monitor with Prometheus / Grafana (Scrape metrics from cAdvisor / Kubelet)
  
```

→ Ensure stable memory usage & no more OOMKills



BEST PRACTICES

- ★ Always set resource requests and limits.
- ★ Avoid setting memory limit too close to actual usage.
- ★ Use VPA to right-size workloads.
- ★ Monitor memory usage continuously.
- ★ Detect and fix memory leaks early.
- ★ Enable liveness & readiness probes.
- ★ Scale horizontally for traffic spikes (HPA).
- ★ Use memory efficient base images.
- ★ Perform load tests before production.



MONITORING METRICS

- `container_memory_usage_bytes`
- `container_memory_working_set_bytes`
- `container_memory_failcnt`
- `oom_kills_total`



EVENT EXAMPLE

```

Warning OOMKilled 10m (x5 over 20m)
Container my-app was killed because it exceeded its memory limit.
  
```



EXIT CODE

```

Exit Code: 137
Reason: OOMKilled
(128 + 9) → Killed by SIGKILL
  
```



MEMORY MANAGEMENT TIPS



Set Appropriate Limits



Optimize Code & Dependencies



Use Efficient Caching



Monitor Continuously



Test Under Load

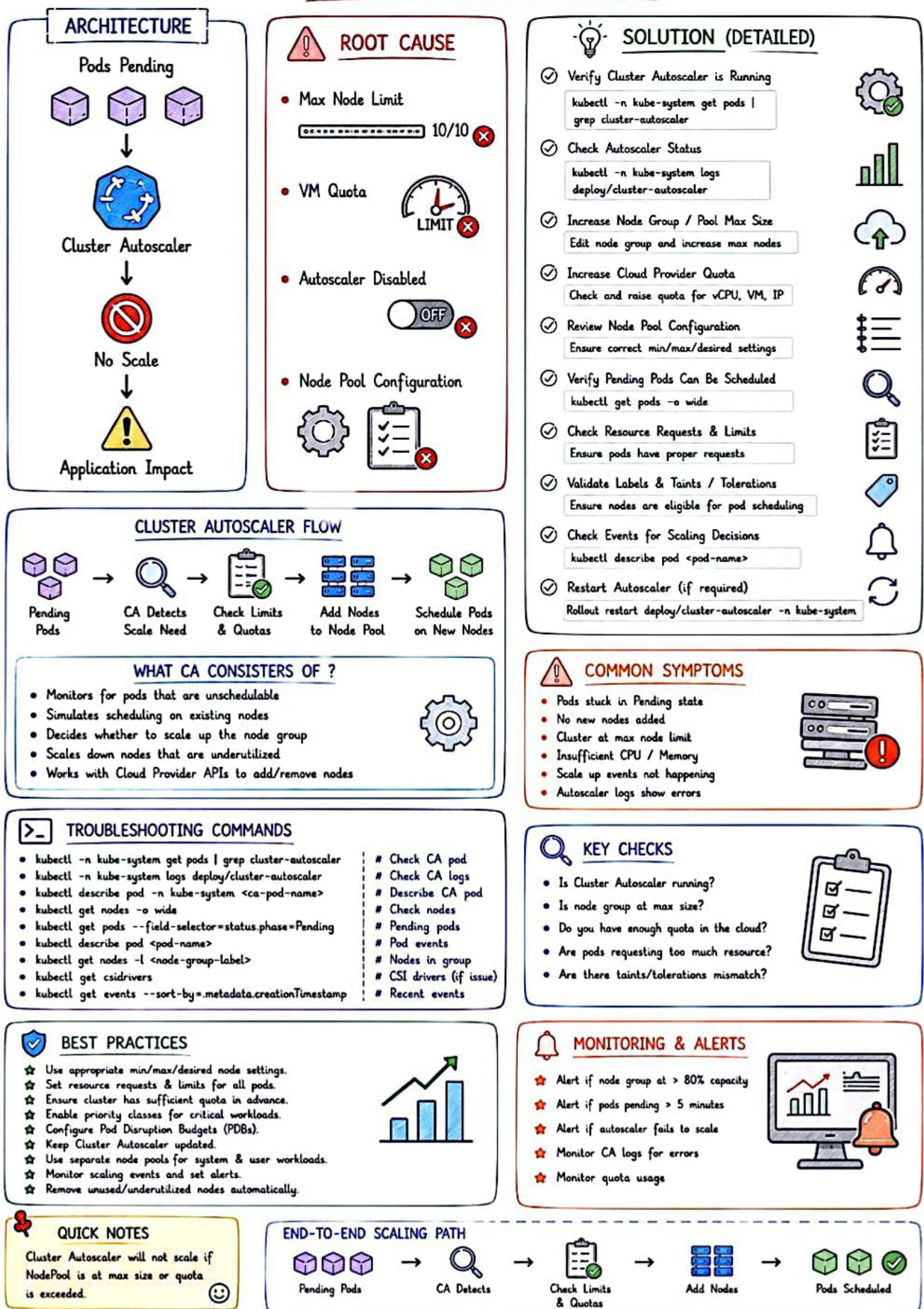


Scale Smartly



kubernetes

15. Cluster Autoscaler Not Scaling





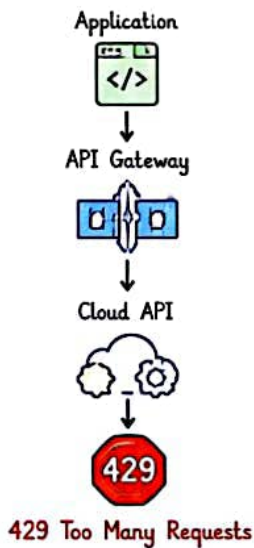
kubernetes





17. API Rate Limiting / Throttling

ARCHITECTURE



ROOT CAUSE

- High Request Rate
Too many requests in a short time
- API Quota Exceeded
Request limit / quota exceeded
- Retry Storm
Aggressive retries causing load amplification
- Burst Traffic
Sudden spike in traffic beyond allowed limit



SOLUTION (DETAILED)

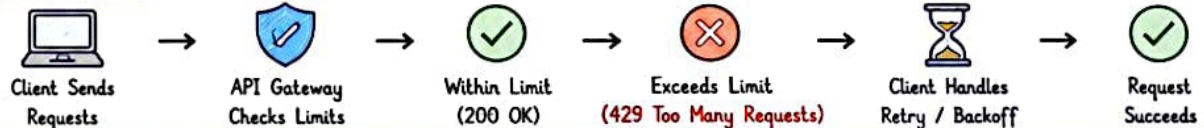
- Implement Exponential Backoff
 - Increase wait time between retries
 - Add jitter to avoid synchronized retries
- Increase API Quota / Limit
 - Contact provider / raise quota
 - Use higher plan if required
- Apply Rate Limiting
 - Limit requests per user/IP/Service
 - Use Token Bucket or Leaky Bucket algorithm
- Enable Request Caching
 - Cache GET responses
 - Use Redis / CDN / In-Memory cache
- Optimize & Batch Requests
 - Reduce unnecessary calls
 - Batch multiple operations
- Monitor & Alert
 - Track 429 errors
 - Set up alerts before hitting limits



Retry after
1s, 2s, 4s, 8s...



HOW IT WORKS



COMMON SYMPTOMS

- HTTP 429 Too Many Requests
- Requests failing intermittently
- API calls delayed or dropped
- Application performance degraded
- Users facing errors during peak time



TROUBLESHOOTING COMMANDS

- `curl -I https://api.example.com/endpoint` # Check response headers
- `curl -v https://api.example.com/endpoint` # View rate limit headers
- Check headers:
X-RateLimit-Limit, X-RateLimit-Remaining, X-RateLimit-Reset, Retry-After # Common rate limit headers
- Check application logs for 429 errors # Search logs
- Monitor API gateway / WAF / CDN logs # Identify spikes

SOLUTION - IMPLEMENTATION EXAMPLES

1 Exponential Backoff (Example)

```
for (int i = 0; i < maxRetries; i++) {
  try {
    callApi();
    break;
  } catch (429) {
    int delay = (int) Math.pow(2, i) * 1000;
    delay += random(0, 1000); // jitter
    sleep(delay);
  }
}
```

Retry with
increasing delay
+ jitter



2 Token Bucket Algorithm (Concept)



3 Caching Example (Redis)

```
GET /products/123
→ Check Redis
→ If hit
→ If miss
  Return cached response
  Call API
  Store in Redis (TTL)
```



BEST PRACTICES

- Know your API limits and monitor usage.
- Use exponential backoff with jitter for retries.
- Avoid unnecessary API calls. Use caching.
- Use batching to reduce the number of requests.
- Implement distributed rate limiting for microservices.
- Use CDN for static and frequently accessed data.
- Set alerts at 79-80% of quota usage.
- Review and clean up unused API calls.



MONITORING & ALERTS

- Monitor 429 error rate & API latency.
- Alert when remaining quota < 20%.
- Track top endpoints by request volume.
- Visualize API usage in Grafana / Kibana / CloudWatch.



USEFUL HEADERS (EXAMPLE)

Header	Description
X-RateLimit-Limit	Total allowed requests in window
X-RateLimit-Remaining	Remaining requests in window
X-RateLimit-Reset	Time when the limit resets (epoch)
Retry-After	Seconds to wait before retry

TRAFFIC FLOW - WITH RATE LIMITING



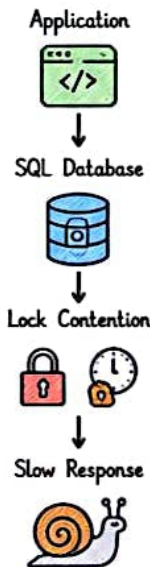
TOOLS & SERVICES





18. Database Deadlock / Slow Queries

ARCHITECTURE



ROOT CAUSE

- **Deadlock**
Two or more transactions waiting for each other's locks.
- **Missing Index**
Queries scan more rows causing high CPU & I/O.
- **Long Transactions**
Transactions holding locks for a long time.
- **Blocking Queries**
One query waiting because another holds the lock.



SOLUTION (DETAILED)

- ✓ **Kill Blocking Session**
 - Identify blocking & blocked sessions.
 - Terminate the blocking session safely.
- ✓ **Optimize Queries**
 - Analyze execution plan.
 - Rewrite inefficient queries.
 - Avoid SELECT * and excessive joins.
- ✓ **Create Indexes**
 - Add proper indexes on WHERE, JOIN, ORDER BY columns.
 - Remove unused or duplicate indexes.
- ✓ **Tune Database**
 - Update statistics regularly.
 - Configure memory, max connections, tempdb (SQL Server) / shared_buffers (PostgreSQL).
 - Enable query cache / plan cache.
- ✓ **Monitor & Alert**
 - Set up alerts for slow queries & deadlocks.
 - Monitor wait stats & lock waits.



HOW IT HAPPENS



COMMON SYMPTOMS

- Queries taking too long to complete
- High CPU / Memory on database
- Blocking / Blocked sessions increasing
- Deadlock errors in application logs
- Time-outs / 504 Gateway Timeout
- Poor application performance



TROUBLESHOOTING COMMANDS

SQL Server

```

-- Blocking sessions
EXEC sp_who2

-- Deadlock graph (Extended Events)
SELECT * FROM sys.dm_tran_locks

-- Top slow queries
SELECT TOP 20 * FROM sys.dm_exec_query_stats
ORDER BY total_elapsed_time DESC

-- Kill session
KILL <spid>
  
```

PostgreSQL

```

-- Blocking sessions
SELECT * FROM pg_locks;

-- Long running queries
SELECT * FROM pg_stat_activity
WHERE state = 'active';

-- Kill session
SELECT pg_terminate_backend(<pid>);

-- Deadlock logs (check logs)
/var/log/postgresql/postgresql.log
  
```

SOLUTION - STEP BY STEP

1 Identify Blocking / Blocked Sessions	Find who is blocking whom.	sp_who2 / pg_locks
2 Kill Blocking Session (if safe)	Terminate the blocking session.	KILL <spid> / pg_terminate_backend()
3 Analyze Execution Plan	Check why query is slow.	Actual Execution Plan (CTRL + M in SSMS)
4 Optimize Queries	Rewrite query / avoid full scans.	Use EXPLAIN / ANALYZE
5 Create Missing Indexes	Add indexes based on query plan.	CREATE INDEX ...
6 Update Statistics	Keep optimizer statistics up-to-date.	UPDATE STATISTICS ...
7 Monitor & Tune	Tune DB settings & monitor.	Wait stats, I/O, Memory

BEST PRACTICES

- ★ Always use proper indexing strategy.
- ★ Avoid long running transactions.
- ★ Commit transactions as early as possible.
- ★ Use connection pooling.
- ★ Regularly update statistics.
- ★ Avoid SELECT * and unnecessary joins.
- ★ Monitor DB performance continuously.
- ★ Use Query Store / Performance Insights.



DEADLOCK EXAMPLE



SLOW QUERY EXAMPLE

```

SELECT * FROM Orders
WHERE OrderDate BETWEEN
'2024-01-01' AND '2024-12-31'
ORDER BY CustomerId;
  
```

Without Index → Full Table Scan
With Index → Fast Execution

KEY METRICS TO MONITOR

- Average Query Duration
- Slow Query Count
- Deadlock Count
- Blocking Sessions Count
- CPU / Memory Usage
- Disk I/O / TempDB Usage



TOOLS

- SQL Server: SSMS, Query Store, Extended Events
- PostgreSQL: pgAdmin, pg_stat_statements extension
- Cloud: Azure Monitor, RDS Performance Insights, Cloud SQL Insights
- APM: Dynatrace, New Relic, Datadog

QUICK REFERENCE FLOW



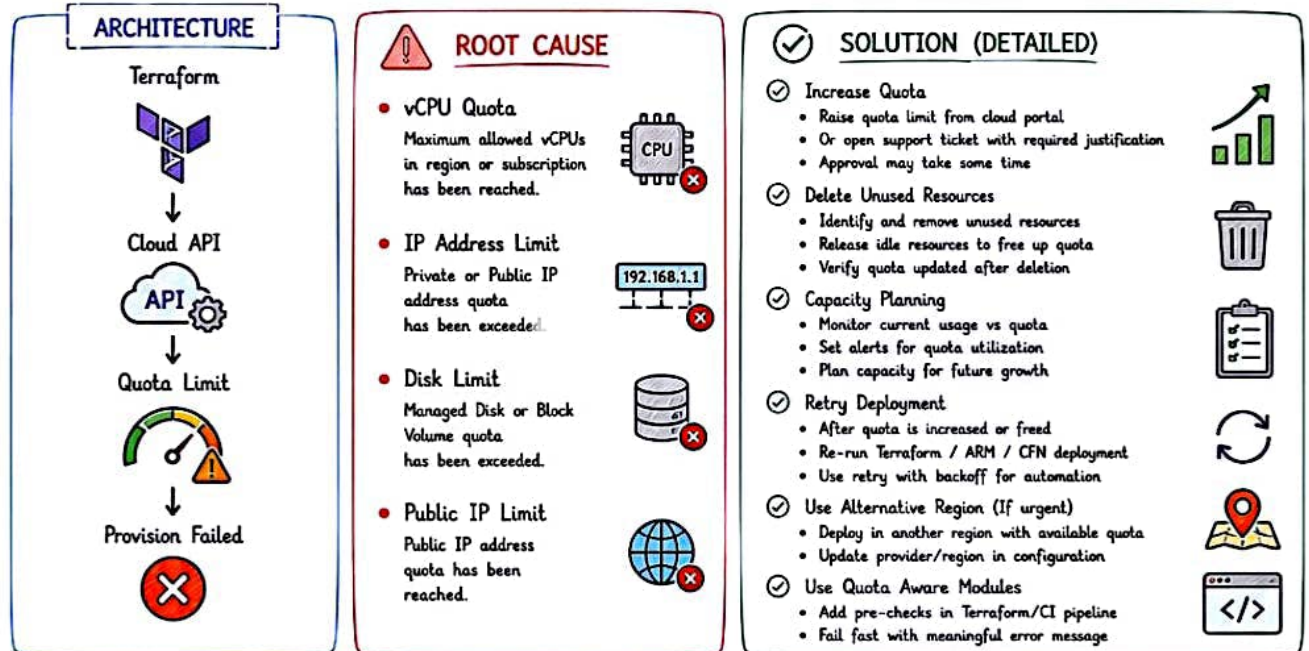
REMEMBER

- Lock less, Index more.
- Tune today, Scale tomorrow.
- Monitor always, Avoid surprises.



kubernetes

19. Cloud Resource Quota Exceeded



COMMON SYMPTOMS

- Error during resource creation
- Provisioning stuck or fails
- Quota exceeded error message
- Partial resources created
- CI/CD pipeline deployment fails

TROUBLESHOOTING COMMANDS

Azure
`az vm list-usage --location <region>`
`az network list-usage --location <region>`
`az disk list-usage --location <region>`

AWS
`aws service-quotas list-aws-default-quotas --region <region>`
`aws service-quotas get-service-quota \`
`--service-code ec2 --quota-code L-1216C47A`

GCP
`gcloud compute regions describe <region> \`
`--format="value(quotas)"`

Check Quota In Cloud Console

- Azure Cost Management + Quotas
- AWS Service Quotas
- GCP IAM & Admin > Quotas

SOLUTION - STEP BY STEP

- Check Current Quota & Usage
- Identify Resource Blocking Deployment
- Increase Quota / Request Limit
- Clean Unused Resources
- Validate Quota Updated
- Update IaC (If Needed)
- Retry Deployment

BEST PRACTICES

- Monitor quota usage regularly.
- Set alerts at 70-80% quota usage.
- Plan capacity in advance.
- Use right-sized resources.
- Clean up unused resources.
- Use multiple regions if possible.
- Document quota requirements.

QUOTA TYPES (EXAMPLES)

Resource Type	Example Quota
vCPU	Standard Dv5 Family vCPUs
Public IP	Public IP Addresses
Private IP	Private IP Addresses per NIC
Managed Disk	Standard HDD Data Disks
Load Balancer	Load Balancer Rules
Network Security Group	Security Rules
Cloud NAT / Gateway	NAT Gateway Ports

QUICK FIX OPTIONS

- Try Alternative Region
Deploy in region with available quota
- Reduce Resource Size
Use smaller instance type or fewer resources
- Schedule Off-Peak Deployments
Avoid peak time quota pressure
- Add Quota Check in Pipeline
Fail fast before deployment

PREVENTION & MONITORING

- Enable quota alerts & notifications
- Use CloudWatch / Azure Monitor / GCP Monitoring
- Tag resources & track ownership
- Review quotas during design phase
- Maintain quota documentation
- Regular clean-up & optimization





kubernetes

20. Terraform Drift / Infrastructure Drift

ARCHITECTURE



ROOT CAUSE

- **Manual Portal Changes**
Resources are modified manually in the cloud portal / CLI.
- **State File Mismatch**
Terraform state file is out of sync with actual infrastructure.
- **Resource Import Missing**
Existing resources are not imported into Terraform state.
- **Parallel Changes**
Multiple users / pipelines make changes without coordination.



SOLUTION (DETAILED)

- ✓ **Run terraform plan**
 - Detect drift between state & real infra.
 - Review changes before applying.
- ✓ **Import Resources**
 - Import existing resources not managed by Terraform.
 - `terraform import <resource> <id>`
- ✓ **Refresh State**
 - Run `terraform refresh` (or `plan`) to update state with actual infrastructure.
 - Ensures state reflects real-world resources.
- ✓ **Restrict Manual Changes**
 - Enforce changes only via Terraform.
 - Use RBAC / Policy to restrict portal access.
- ✓ **Enforce IaC Governance**
 - Code reviews, approvals & CI/CD checks.
 - Use Sentinel / OPA / Conftest for policy.
 - Maintain audit & change tracking.

HOW DRIFT HAPPENS (FLOW)



COMMON SYMPTOMS

- terraform plan shows unexpected changes
- Resources differ from what exists in cloud
- Updates / destroys not reflected in state
- CI/CD pipelines show drift warnings
- Manual changes not tracked in code

Sample terraform plan output

```

# aws_instance.web_server
- instance_type = "t2.micro" -> "t2.medium"
- tags = {
  - "env" = "dev"
  + "env" = "prod"
}
~/.aws/security-group.sg (forces new resource)
    
```



TROUBLESHOOTING COMMANDS

- `terraform plan`
- `terraform plan --refresh-only`
- `terraform refresh`
- `terraform state list`
- `terraform state show <resource>`
- `terraform import <resource> <id>`
- `terraform state rm <resource>`
- `terraform graph | dot -Tpng > graph.png`
- Detect drift
- Refresh state only
- Update state from real infra
- List resources in state
- Show resource in state
- Import existing resource
- Remove from state (if needed)
- Visualize dependencies



SOLUTION - STEP BY STEP

- 1 Detect Drift
`terraform plan` → Identify unexpected changes
- 2 Refresh State
`terraform plan --refresh-only` or `terraform refresh` → Sync state with real infra
- 3 Review Changes
`terraform plan` → Review what Terraform wants to change
- 4 Import Missing Resources
`terraform import <type.name> <resource-id>` → Bring existing resources under Terraform control
- 5 Apply Changes
`terraform apply` → Apply required changes safely
- 6 Prevent Future Drift
Restrict manual access
Enforce IaC workflow → Ensure IaC is the single source of truth

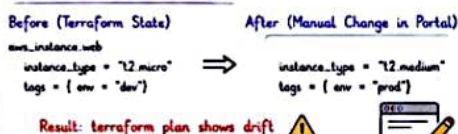


BEST PRACTICES

- ★ Enforce all changes through Terraform (IaC).
- ★ Restrict cloud console access using RBAC / Policies.
- ★ Use remote state backend with locking (S3 + DynamoDB).
- ★ Enable drift detection in CI/CD pipelines.
- ★ Schedule regular terraform plan in pipelines.
- ★ Use policy as code (Sentinel / OPA / Conftest).
- ★ Tag all resources with owner / env / cost-center.
- ★ Maintain documentation & change history.



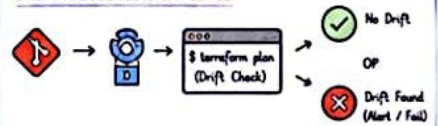
EXAMPLE: MANUAL CHANGE



EXAMPLE: IMPORT RESOURCE



DRIFT DETECTION IN CI/CD



DRIFT PREVENTION CONTROLS

- RBAC & Least Privilege
- Service Principals for IaC
- Azure Policy / AWS SCPs to restrict changes
- Audit logs & alerts



KEY METRICS TO TRACK

- Number of drift detections
- Drift resolution time
- Unauthorized change attempts
- IaC compliance percentage
- Resources not managed by Terraform



TOOLS THAT HELP

- Terraform Cloud / Enterprise (Drift Detection)
- Spacelift / Atlantis / Env0
- AWS Config / Azure Policy / GCP Config
- OPA / Sentinel / Conftest
- CloudTrail / Activity Logs / Audit Logs



IaC GOVERNANCE GOLDEN RULE

</> Code is Source of Truth



Automate Everything



Detect Drift Early



Review & Approve



Monitor & Audit



Secure & Restrict Access



Document & Improve